

Universidad San Carlos de Guatemala

Centro universitario de Occidente

División de Ciencias de la ingeniería

Ingeniería en Ciencias y Sistemas

Estructura De Datos

Ing. Mario Ramirez Tobar



TADs
PROYECTO 1 FASE 1
GESTIÓN DE CATALOGO DE PRODUCTOS DE SUPERMERCADO

Cristian Alejandro Roldán López
Carné: 202147280

Quetzaltenango 03 de abril de 2026

Producto

Interfaz Pública

```
Producto();  
Producto(const string &nombre, const string &codigo,  
         const string &categoria, const string  
         &fechaCaducidad,  
         const string &marca, const double &precio,  
         const int &cantidad);  
string getNombre() const;  
string getCodigo() const;  
string getCategoria() const;  
string getFechaCaducidad() const;  
string getMarca() const;  
double getPrecio() const;  
int getCantidad() const;  
void setNombre(const string &nombre);  
void setCodigo(const string &codigo);  
void setCategoria(const string &categoria);  
void setFechaCaducidad(const string &caducidad);
```

Interfaz Privada

```
string nombre;  
string codigo;  
string categoria;  
string fechaCaducidad;  
string marca;  
double precio;  
int cantidad;
```

Árbol AVL

Interfaz Pública

```
ArbolAvl();  
void insert(Producto producto);  
Producto *buscar(string nombre);  
void imprimir();  
void eliminar(string nombre);  
NodoAvl *getRaiz() const { return raiz; }  
NodoAvl *eliminar(NodoAvl *nodo, string nombre);  
NodoAvl *nodoMinimo(NodoAvl *nodo);  
void generarDotRecursivo(NodoAvl *nodo, fstream  
                        &archivo);  
void generarDOT(string nombreArchivo);
```

Interfaz Privada

```
NodoAvl *raiz;  
int altura(NodoAvl *nodo);  
int balance(NodoAvl *nodo);  
NodoAvl *rotacionDerecha(NodoAvl *y);  
NodoAvl *rotacionIzquierda(NodoAvl *x);  
NodoAvl *insertar(NodoAvl *nodo, Producto producto);  
NodoAvl *buscar(NodoAvl *nodo, string nombre);  
void inOrden(NodoAvl *nodo);
```

Nodo AVL

Interfaz Pública

```
NodoAvl();  
NodoAvl(Producto producto);  
int getAltura();  
NodoAvl *getIzquierda();  
NodoAvl *getDerecha();  
void setIzquierda(NodoAvl* nodo);  
void setDerecha(NodoAvl* nodo);  
void setProducto(const Producto& producto);  
void setAltura(int altura);  
Producto getProducto();
```

Interfaz Privada

```
Producto producto;  
NodoAvl *derecha;  
NodoAvl *izquierda;  
int altura;  
NodoAvl *insertar(NodoAvl *nodo, Producto producto);  
NodoAvl *buscar(NodoAvl *nodo, string nombre);  
void inOrden(NodoAvl *nodo);
```

Árbol B

Interfaz Pública

```
ArbolB();  
ArbolB(int gradoMinimo);  
~ArbolB();  
void insertar(Producto producto);  
Producto *buscarFecha(string fecha);  
void buscarRangoFecha(string fechaInicio, string  
    fechaFin);  
void imprimirOrdenado();  
void generarDOT(string nombreArchivo);  
void eliminar(string fecha);  
void eliminarRecursivo(NodoB *nodo, int clave);  
void eliminarDeHoja(NodoB *nodo, int indice);  
void eliminarDeInterno(NodoB *nodo, int indice);  
Producto obtenerPredecesor(NodoB *nodo, int indice);  
Producto obtenerSucesor(NodoB *nodo, int indice);  
void llenar(NodoB *nodo, int indice);  
void fusionar(NodoB *nodo, int indice);  
void pedirPrestadoAnterior(NodoB *nodo, int indice);  
void pedirPrestadoSiguiente(NodoB *nodo, int indice);  
NodoB *getRaiz() const { return raiz; }
```

Interfaz Privada

```
NodoAvl *raiz;  
int altura(NodoAvl *nodo);  
int balance(NodoAvl *nodo);  
NodoAvl *rotacionDerecha(NodoAvl *y);  
NodoAvl *rotacionIzquierda(NodoAvl *x);  
NodoAvl *insertar(NodoAvl *nodo, Producto producto);  
NodoAvl *buscar(NodoAvl *nodo, string nombre);  
void inOrden(NodoAvl *nodo);
```

Nodo B

Interfaz Pública

```
NodoB(int gradoMinimo, bool esHoja);  
    ~NodoB();  
    int getCantidadClaves();  
    bool hoja();  
    Producto* getClave(int indice);  
    NodoB* getHijo(int indice);  
void setClave(int indice, Producto producto);  
void setHijo(int indice, NodoB* nodo);  
    void incrementClave();  
    void decrementClave();
```

Interfaz Privada

```
Producto* clave;  
NodoB** hijos;  
int gradoMinimo;  
int cantidadClaves;  
bool esHoja;
```

Árbol B+

Interfaz Pública

```
ArbolBMas();  
ArbolBMas(int orden);  
~ArbolBMas();  
void insertar(const Producto &p);  
void eliminar(const string &categoria, const string  
             &codigoBarra);  
void buscarCategoria(const string &categoria) const;  
void imprimirOrdenado() const;  
void mostrarArbol() const;  
void generarDOT(const string &nombreArchivo);
```

Interfaz Privada

```
NodoBMas *raiz;  
NodoBMas *primeraHoja;  
int orden;  
void insertarEnNodo(NodoBMas *nodo, const string  
                  &categoria, const Producto &p);  
void dividirHijo(NodoBMas *padre, int indice,  
                NodoBMas *hijo);  
void dividirHojaHijo(NodoBMas *padre, int indice,  
                    NodoBMas *hijo);  
void eliminarRecursivo(NodoBMas *nodo, const string  
                    &categoria, const string &codigoBarra);  
void eliminarDeHoja(NodoBMas *nodo, int idx);  
void eliminarDeInterno(NodoBMas *nodo, int idx);  
void llenar(NodoBMas *nodo, int idx);  
void pedirPrestadoAnterior(NodoBMas *nodo, int idx);  
void pedirPrestadoSiguiente(NodoBMas *nodo, int idx);  
void fusionar(NodoBMas *nodo, int idx);  
void generarDotRecursivo(NodoBMas *nodo, fstream  
                        &archivo, int &contadorHoja);  
void liberarNodo(NodoBMas *nodo);  
void imprimirNivelPorNivel() const;
```

Nodo B+

Interfaz Pública

```
NodoBMas(int orden, bool esHoja);
~NodoBMas();
bool getEsHoja();
int getNumClaves();
int getOrden();
string getClave(int i);
Producto getProducto(int i);
NodoBMas *getHijo(int i);
NodoBMas *getSiguiente();
NodoBMas *getAnterior();
void setClave(int i, string clave);
void setProducto(int i, const Producto &p);
void setHijo(int i, NodoBMas *nodo);
void setSiguiente(NodoBMas *nodoSiguiente);
void setAnterior(NodoBMas *nodoAnterior);
void incrementarClaves();
void decrementarClaves();
void setNumClaves(int n);
bool lleno() const;
```

Interfaz Privada

```
string *claves;
int numClaves;
Producto *producto;
NodoBMas **nodoHijo;
bool esHoja;
int orden;
NodoBMas *siguiente;
NodoBMas *anterior;
```


Lista Enlazada

Interfaz Pública

```
ListaEnlazada() ;  
~ListaEnlazada() ;  
void insert(Producto producto) ;  
void remove(Producto producto) ;  
void eliminar(string codigo) ;  
void buscar(string nombre) ;  
void imprimir() ;
```

Interfaz Privada

```
Nodo *head;
```

Nodo

Interfaz Pública

```
Nodo() ;  
Nodo(Producto p) ;  
Producto getProducto() ;  
void setProducto(Producto p) ;  
void setSiguiente(Nodo *p) ;  
Nodo *getSiguiente()
```

Interfaz Privada

```
Producto producto;  
Nodo *siguiente;
```

Cargar CSV

Interfaz Pública

```
CargarCSV(const string &rutaArchivo, const string
          &rutaLog = "errors.log");
int cargar(ArbolB *arbolB, ArbolAvl *arbolAVL,
          ArbolBMas *arbolBMas, ListaEnlazada *lista);
```

Interfaz Privada

```
string rutaArchivo;
string rutaLog;
string quitarComillas(const string &campo);
bool parsearLinea(const string &linea, string campos[], int
numCampos);
void loggear(ofstream &log, int numLinea, const string &mensaje,
const string &linea);
bool esNumerico(const string &s);
bool esEntero(const string &s);
void rollback(const Producto &p,
              ArbolB *arbolB,
              ArbolAvl *avl,
              ArbolBMas *arbolBMas,
              ListaEnlazada *lista,
              bool insertadoB,
              bool insertadoBMas,
              bool insertadoAVL,
              bool insertadoHash,
              bool insertadoLista);
```